King Saud University
College of Computer and Information Sciences
Department of Information Technology

IT362: Principles of Data Science

Semester-2, 1447H

Phase 2: Data Processing, Cleaning, and Exploratory Data Analysis (EDA)

Logbook

Topic: Sentiment analysis of reviews of beauty centres

Prepared by

Section No	Name	ID
73059	Leen Almuqhim	445201272
	Remas Alayiedh	445202140
	Lubna Alqifari	445202104
	Aljoury Alreemi	445200074
	Khadijah Alshehri	445202138

Supervised by: Lama Alsudias

PRIMARY DATA Acquisition

Step 1: Primary Data Acquisition

What was done:

Primary data was collected using the Google Places API, which provides customer reviews, star ratings, timestamps, and business identifiers in structured format.

Why it was necessary:

No publicly available dataset was found that directly targets beauty center reviews specific to the research objective. Therefore, primary data collection was required to obtain domain-specific service-based reviews suitable for sentiment and behavioral analysis.

How it was implemented:

Google Reviews data was exported and stored in CSV format. The dataset was loaded into Google Colab using Pandas for structured inspection and preprocessing.

Code Snippet:

```
df = pd.read_csv("/content/unstructured_reviews_readable (1).csv")
```

Challenges:

• Raw timestamps required conversion to readable date format.

• Minor structural inconsistencies required inspection before analysis.

Solution:

• Converted Unix timestamps to datetime format.

• Performed structural validation before preprocessing.

Data processing and Cleaning

The raw dataset was inspected to understand its structure. Key fields included review text, star ratings, timestamps, and business identifiers, in addition to descriptive fields such as language and relative time.

Step 1: Data processing and Cleaning

What was done: The dataset columns were renamed to align with the structure of the secondary dataset. Checed for duplicates record. The date column was formatted properly for time-based analysis.

Why it was necessary: Standardizing column names ensures compatibility for later comparison between primary and secondary datasets. Cleaning ensures data integrity and reliability.

How it was implemented: Column renaming was performed using Pandas. Indexes were reset and a unique review_id column was created. The cleaned dataset was stored in a structured format.

Handling Missing Text: Missing and empty review texts were checked and removed if found. One record with empty or missing text was removed (2030 → 2029),indicating no missing/blank texts in this file.

```
before_shape = primary_clean.shape

primary_clean = primary_clean.dropna(subset=["text"])
primary_clean =
primary_clean[primary_clean["text"].astype(str).str.strip() != ""]

after_shape = primary_clean.shape

before_shape, after_shape
```

Duplicates Records: Number of duplicates detected: 0

```
dup_rows = primary_clean.duplicated(subset=["business_id", "text",
dup_rows
```

Validating Ratings: Ratings were coerced to numeric to handle invalid entries. Out-of-range values were filtered to enforce the 1–5 scale. After validation.

```
primary_clean["stars"] = pd.to_numeric(primary_clean["stars"],
errors="coerce")

primary_clean = primary_clean[primary_clean["stars"].between(1, 5)]
primary_clean["stars"] = primary_clean["stars"].astype(int)
```

```
primary_clean["stars"].dtype, primary_clean["stars"].unique()
```

Sentiment Label Generation:

Sentiment labels were derived from star ratings to support sentiment distribution analysis and enable comparison with the secondary dataset.

```
# Define sentiment mapping function
def map_sentiment(stars):
    if stars <= 2:
        return "Negative"
    elif stars == 3:
        return "Neutral"
    else:
        return "Positive"
        # Apply sentiment mapping
primary_clean["sentiment_label"] = (
    primary_clean["stars"]
    .apply(map_sentiment)
)

# Preview sentiment results
primary_clean[["stars", "sentiment_label"]].head()
```

Creating Review Length Feature:

A new feature, review_length, was created to calculate the number of words in each review. This enables quantitative analysis of engagement patterns and supports further NLP and modeling tasks.

```
primary_clean["review_length"] = primary_clean["text"].apply(lambda x:
len(str(x).split()))

primary_clean[["text", "review_length"]].head()
```

Timestamp Conversion: Timestamps were converted to datetime. Since timestamps may appear in seconds or milliseconds, the unit was inferred automatically using a median threshold (>= 1e12 → milliseconds) to avoid incorrect date parsing.

```
ts = pd.to_numeric(primary_clean["date"], errors="coerce")

unit = "ms" if ts.dropna().astype(float).median() >= 1e12 else "s"

primary_clean["date"] = pd.to_datetime(ts, unit=unit, errors="coerce")

primary_clean["date"].head(), primary_clean["date"].dtype
```

Light Text Normalization: Light normalization was applied to standardize text before further NLP steps (lowercasing, URL removal, whitespace normalization)

```
def clean_text_func(text):
    text = str(text).lower()
    text = re.sub(r"http\S+", "", text)    # remove URLs
    text = re.sub(r"\s+", " ", text)       # normalize whitespace
    return text.strip()

primary_clean["clean_text"] = primary_clean["text"].apply(clean_text_func)

primary_clean[["text", "clean_text"]].head()
```

Code Snippet:

```
primary_clean = df.rename(columns={
    "place_id": "business_id",
    "review_text": "text",
    "rating": "stars",
    "time": "date"
}).copy()
primary_clean.head(), primary_clean.shape
```

```
primary_clean = primary_clean.reset_index(drop=True)
primary_clean["review_id"] = primary_clean.index.astype(str)

final_primary = primary_clean[
    ["review_id", "business_id", "stars", "sentiment_label", "clean_text",
"processed_text", "date"]
].copy()

final_primary.head(), final_primary.shape
```

Challenges: Inconsistent schema (different column names) and ambiguous timestamp units.

Solution: : Standardized column names and inferred timestamp units programmatically before converting to datetime

Step 2: Advanced Text Processing Techniques

What was done:

Advanced NLP preprocessing techniques were applied, and a new column (processed_text) was created while preserving the original text.

Techniques Implemented:

• Tokenization

• Stopword removal

• Punctuation and number removal

• Lemmatization

Why it was necessary:

• Reduce textual noise

• Normalize vocabulary

• Prepare data for machine learning

• Improve feature quality for vectorization

How it was implemented:

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
```

Processing function:

```
import re
def advanced_preprocess(text):
    # Lowercas
    text = str(text).lower()
    # Remove URLs
    text = re.sub(r'http\S+', '', text)
    # Remove punctuation & numbers
    text = re.sub(r'[^a-z\s]', ' ', text)
    # Tokenization
    tokens = word_tokenize(text)
    # Remove stopwords
    tokens = [word for word in tokens if word not in stop_words]
    # Lemmatization
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    # Join back
    return " ".join(tokens).strip()
```

Challenges:

Risk of over-cleaning and potential loss of semantic meaning.

Stopword removal was limited to English language resources.

Solution:

- The original text column was retained to ensure no raw information was lost.
- Lemmatization was preferred over stemming to preserve semantic meaning.
-

Step 3: Text Vectorization Preparation

What was done:

The processed_text column was transformed into numerical feature representations using Bag of Words(BoW), (CountVectorizer) and TF-IDF vectorization.

Why it was necessary:

Text data must be converted into numerical feature vectors before being used in classification or predictive modeling tasks.

How it was implemented:

BoW:

```
bow_vectorizer = CountVectorizer(max_features=5000) X_bow =
bow_vectorizer.fit_transform(primary_clean['processed_text']) X_bow.shape
```

TF-IDF:

```
tfidf_vectorizer = TfidfVectorizer(max_features=5000) X_tfidf =
tfidf_vectorizer.fit_transform(primary_clean['processed_text'])
X_tfidf.shape
```

Design Decision:

The vocabulary size was limited to 5000 features to control dimensionality and reduce computational cost.

Step 4: Dataset Storage

What was done:

A final structured version of the cleaned primary dataset was created by selecting relevant engineered features, generating a unique review_id, and separating it from the intermediate processing dataset before saving it as a CSV file.

Why it was necessary:

Creating a finalized structured dataset ensures reproducibility, prevents accidental modification of intermediate data, and provides a clean, consistent input for EDA and future modeling tasks

Creating Final Structured Dataset:

```
primary_clean = primary_clean.reset_index(drop=True)
primary_clean["review_id"] = primary_clean.index.astype(str)

final_primary = primary_clean[
    ["review_id", "business_id", "stars", "sentiment_label", "clean_text",
"processed_text", "date"]
].copy()

final_primary.head(), final_primary.shape
```

Saving Dataset:

```
final_primary.to_csv("primary_google_beauty_reviews_cleaned.csv",
index=False)
```

Primary Data (EDA)

During this phase, exploratory data analysis was conducted on the primary Google beauty reviews dataset to identify structural patterns, sentiment behavior, textual characteristics, and temporal rating trends.

Objective:

The objective was to understand rating distribution, sentiment proportions, review length variability, correlation structure, textual patterns, and time-based rating trends.

Steps Taken:

1. Loaded and inspected dataset structure and variable types.
2. Generated descriptive statistics for star ratings and review length.
3. Created bar and pie charts to analyze star rating distribution.
4. Generated histogram and ECDF plots to examine review length distribution and skewness.
5. Created boxplots to analyze the relationship between review length and star rating.
6. Computed and visualized a correlation matrix between rating and review length.
7. Performed time-series analysis to examine average rating trends over time.
8. Applied CountVectorizer to extract the top 20 most frequent words in processed reviews.
9. Generated and analyzed the top 10 bigrams (two-word combinations).

10. Applied TF-IDF vectorization to identify the most distinctive and informative textual terms.

Types of Analysis Performed:

- Descriptive statistical analysis ·
- Distribution analysis
- Relational and correlation analysis ·
- Time-series trend analysis · Text frequency analysis (unigram and bigram) ·
- TF-IDF feature importance analysis.

Tools Used:

- Python
- Pandas
- Matplotlib
- Seaborn
- NLTK
- Scikit-learn

Key Findings:

• 5-star reviews account for approximately 77.4% of the dataset, indicating a strong class imbalance toward positive sentiment

• Review length follows a right-skewed distribution with several outliers.

• A moderate negative correlation (r = -0.37) exists between star rating and review length.

• Early fluctuations may be influenced by observations in certain months..

• Text frequency analysis shows dominance of experience-related terms such as "time", "experience", "recommend", and "treatment".

• Bigram analysis highlights recurring positive expressions such as "highly recommend" and "customer service", reinforcing the overall positive sentiment trend.

• TF-IDF results emphasize service-specific and treatment-related terms (e.g., "massage", "treatment", "professional"), indicating that customer feedback centers on service experience rather than generic praise

• The dominance of highly positive textual expressions aligns with the observed rating imbalance toward 5-star reviews.

Challenges:

- Severe class imbalance toward 5-star ratings may influence model generalization and interpretation.
- Extreme review length values required careful visualization scaling.
- Text preprocessing required balancing noise reduction and meaning preservation.
- High frequency of generic positive words required the use of TF-IDF to distinguish meaningful service-related terms from repetitive praise expressions.

Secondary Data Acquisition

Step 1: Secondary Data Acquisition

What was done: Secondary data was collected from the **Yelp Open Dataset**, which provides large-scale customer reviews and business metadata in JSON format.

Why it was necessary: No publicly available dataset was found that directly targets beauty center service reviews for sentiment analysis. Most available datasets focus on products or general services. Therefore, a general service-review dataset was selected and later filtered to match the project's domain.

How it was implemented: Because the dataset files were very large, Google Colab was used, and Google Drive was mounted to access the data efficiently.

Code Snippet:

```
from google.colab import drive
drive.mount('/content/drive')
```

Challenges:

- **Large file size** prevented loading the dataset into memory at once.

Solution:

- Used Google Colab and chunk-based processing to handle the data incrementally.

Step 2: Filtering Beauty-Related Businesses

What was done: The Yelp business dataset was filtered to retain only beauty and personal care service businesses.

Why it was necessary: The original Yelp dataset includes businesses from many unrelated domains (e.g., restaurants, hotels). Filtering was required to ensure that the secondary data represents service-based beauty experiences, making it comparable to the primary dataset.

How it was implemented: The business data was loaded in chunks, and a keyword-based filter was applied to the `categories` column.

Code Snippet:

```
keywords = "Beauty & Spas|Hair Salons|Nail Salons|Skin Care|Cosmetic
Services"
beauty_business_chunks = []
for chunk in pd.read_json(
    business_path, lines=True, chunksize=50_000
):
    filtered = chunk[
        chunk['categories'].str.contains(keywords, na=False)
    ]
    beauty_business_chunks.append(filtered)
beauty_business = pd.concat(beauty_business_chunks)
```

Challenges:

- Some businesses contained multiple categories.

Solution:

- Keyword-based filtering was used to capture all relevant beauty-related services without excluding mixed-category businesses.

Step 3: Filtering Reviews by Business ID

What was done: Customer reviews were filtered to include only those associated with the previously identified beauty-related businesses.

Why it was necessary: This step ensured that all reviews correspond exclusively to beauty and spa services, eliminating irrelevant service reviews.

How it was implemented: Business IDs from the filtered business dataset were extracted and used to filter the review dataset.

Code Snippet:

```
beauty_ids = set(beauty_business['business_id'])
review_path = "/content/drive/MyDrive/yelp_academic_dataset_review.json"
beauty_review_chunks = []
for chunk in pd.read_json(
    review_path,
    lines=True,
    chunksize=100_000
):
    filtered = chunk[
        chunk['business_id'].isin(beauty_ids)
    ]
    beauty_review_chunks.append(filtered)
beauty_reviews = pd.concat(beauty_review_chunks)
beauty_reviews.head()
```

Challenges:

- Large review file size.

Solution:

- Processed reviews in chunks and concatenated filtered results.

Step 4: Data Inspection and Validation

What was done: Sample records from both business and review datasets were inspected.

Why it was necessary: To verify that filtering steps were correctly applied and that the data contained meaningful review text and valid ratings.

How it was implemented: Displayed sample rows and category frequency counts.

Code Snippet:

```
beauty_reviews[['business_id', 'stars', 'text']].head(5)
beauty_business[['name', 'categories']].head(10)
```

Summary and Rationale: This data acquisition process ensured that the secondary dataset consists exclusively of service-based customer reviews from beauty and personal care businesses. Chunk-based processing was essential to manage large file sizes, while keyword-based filtering guaranteed relevance to the project domain. The resulting datasets provide a reliable foundation for secondary exploratory data analysis and comparison with primary beauty center reviews.

Secondary Data Cleaning and Preprocessing

Two raw Yelp datasets were inspected: a reviews dataset containing 10,000 customer reviews and a business dataset containing 14,292 business records. Initial inspection showed that reviews include textual feedback and star ratings, while business data includes service categories used for filtering relevant beauty-related services.

Step 1: Data Cleaning and Preprocessing

What was done: The filtered review dataset was cleaned to ensure data quality.

Why it was necessary: Cleaning was required to prepare the dataset for exploratory data analysis and to avoid biased or misleading results.

Cleaning Steps Performed:

- Checked for missing or empty review text
- Checked for duplicate reviews
- Validated rating values
- Applied light text normalization

How it was implemented:

Handling Missing Text: The review text column was inspected for missing or empty values. No records were removed, indicating that all reviews contained valid textual content.

```
reviews_clean = reviews_filtered.dropna(subset=['text'])
reviews_clean = reviews_clean[reviews_clean['text'].str.strip() != '']
```

Removing Duplicates: The dataset was checked for duplicate reviews based on review IDs and review text. No duplicate records were found, so no entries were removed at this stage.

```
dup_ids = reviews_clean.duplicated(subset='review_id').sum()
dup_texts = reviews_clean.duplicated(subset='text').sum()
```

Validating Ratings: Star ratings were inspected to ensure valid numeric values. All ratings were within the expected 1–5 range, and no invalid entries were detected.

```
reviews_clean = reviews_clean[reviews_clean['stars'].between(1, 5)]
```

Light Text Normalization: Review text was lightly normalized by converting to lowercase, removing URLs, and standardizing whitespace. Advanced NLP preprocessing was intentionally avoided to preserve the natural structure of customer language for exploratory analysis.

```
import re
def clean_text(text):
    text = text.lower()
    text = re.sub(r'http\S+', '', text)  # remove URLs
    text = re.sub(r'\s+', ' ', text)     # normalize whitespace
    return text.strip()
reviews_clean['clean_text'] = reviews_clean['text'].apply(clean_text)
```

Labeling and Transformation: Sentiment labels were generated from star ratings to support exploratory analysis and comparison with primary data. Ratings of 1–2 were labeled as Negative, 3 as Neutral, and 4–5 as Positive.

```
def map_sentiment(stars):
    if stars <= 2:
        return 'Negative'
    elif stars == 3:
        return 'Neutral'
    else:
        return 'Positive'
reviews_clean['sentiment_label'] =
reviews_clean['stars'].apply(map_sentiment)
```

Design Decision: Punctuation marks were intentionally preserved during preprocessing, as they may carry expressive and emotional information useful for exploratory analysis of customer sentiment. Advanced text preprocessing techniques such as stopword removal, stemming, lemmatization, and vectorization were intentionally deferred to later modeling phases, as the focus of this phase is exploratory analysis and understanding the natural structure of customer reviews.

Step 2: Advanced Text Processing Techniques

What was done: Advanced Natural Language Processing (NLP) preprocessing techniques were applied to prepare the dataset for future sentiment modeling. A new column (processed_text) was created while preserving the original and lightly cleaned text. The following techniques were implemented:

- Tokenization
- Stopword removal
- Punctuation and number removal
- Lemmatization

Why it was necessary: Although this phase primarily focuses on data preparation for exploratory analysis, advanced preprocessing was implemented to:

- Reduce noise in textual data
- Normalize vocabulary variations
- Improve future model performance
- Ensure readiness for machine learning tasks
- Demonstrate understanding of core NLP preprocessing techniques

Lemmatization was chosen over stemming to preserve meaningful word forms and maintain interpretability.

How it was implemented: NLTK was used for tokenization, stopword removal, and lemmatization.

```
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('omw-1.4')

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
```

Processing Function:

```
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()
def advanced_preprocess(text):
    text = re.sub(r'[^a-z\s]', '', text)
    # Tokenization
    tokens = word_tokenize(text)
    # Remove stopwords
    tokens = [word for word in tokens if word not in stop_words]
    # Lemmatization
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    # Join back
    return " ".join(tokens)
```

Applying to dataset:

```
reviews_clean['processed_text'] =
reviews_clean['text'].apply(advanced_preprocess)
```

Challenges:

- Over-cleaning risk: Removing too many words (especially negations like "not") can distort sentiment meaning.
- Computational cost: Processing large datasets with tokenization and lemmatization increases runtime.

How Challenges Were Addressed:

- Negation words were preserved in the stopword list to maintain sentiment meaning.
- Processing was applied after filtering to reduce dataset size and improve efficiency.
- The original text column was retained to allow comparison if needed.

Step 3: Text Vectorization Preparation

What was done: Two common text representation techniques were prepared for future modeling: Bag of Words (BoW) and TF-IDF (Term Frequency–Inverse Document Frequency).

Why it was necessary: Machine learning models cannot process raw text directly. Text must be converted into numerical features. Preparing both BoW and TF-IDF representations allows comparison of feature extraction approaches in later phases.

How it was implemented:

BoW:

```
from sklearn.feature_extraction.text import CountVectorizer
bow_vectorizer = CountVectorizer(
    max_features=5000
)
X_bow = bow_vectorizer.fit_transform(
    reviews_clean['processed_text']
)
```

TF-IDF:

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer(
    max_features=5000
)
X_tfidf = tfidf_vectorizer.fit_transform(
    reviews_clean['processed_text']
)
```

Design Decision: Vectorization outputs were generated for experimentation purposes but were not used in this phase for modeling. Their primary purpose is to ensure readiness for the modeling phase.

Step 4: Dataset Storage

What was done: The filtered business and review datasets were saved in CSV format after completing the data acquisition and filtering steps.

Why it was necessary: Saving the filtered datasets in a structured format ensures reproducibility and allows them to be reused consistently during exploratory data analysis and comparison with the primary dataset.

How it was implemented:

```
final_secondary = reviews_clean[
    [
        'review_id',
        'business_id',
        'stars',
        'sentiment_label',
        'clean_text',
        'processed_text',
        'date'
    ]
]
final_secondary.to_csv(
    'secondary_yelp_beauty_reviews_cleaned.csv',
    index=False
)
```

Design Decision: The final secondary dataset includes both lightly cleaned text for exploratory analysis and fully processed text for future modelling tasks.

Secondary Data (EDA)

During this phase, exploratory data analysis was conducted on the secondary Yelp beauty reviews dataset to identify structural patterns, behavioral trends, and relationships among numerical variables.

Objective:

The objective was to understand rating distribution, review length variability, engagement behavior, correlation structure, and temporal rating trends.

Steps Taken:

1. Loaded and inspected the dataset structure and variable types.
2. Generated descriptive statistics for numerical features.
3. Created bar and pie charts to analyze star rating distribution.
4. Generated histogram and ECDF plots to examine review length distribution and skewness.
5. Created boxplots to analyze the relationship between review length and star rating.
6. Computed and visualized a correlation matrix to identify relationships among stars, review length, and engagement metrics.
7. Generated scatter plots to observe useful votes distribution across ratings.
8. Calculated and plotted average useful votes by rating to identify engagement trends.
9. Performed time-series analysis to examine average rating behavior over time.
10. Conducted text-based exploratory analysis using word frequency, bigram analysis, and TF-IDF to identify dominant themes and linguistic patterns in review content.

Types of Analysis Performed:

Descriptive analysis

Distribution analysis

Relational analysis

Correlation analysis

Engagement analysis

Trend (time-series) analysis

Text-based exploratory analysis (word frequency, bigram analysis, TF-IDF)

Tools Used:

Python

Pandas

Matplotlib

Seaborn

Scikit-learn (CountVectorizer, TfidfVectorizer)

Key Findings:

• 5-star reviews represent approximately 60% of the dataset.

• Review length follows a right-skewed distribution with extreme outliers.

• A moderate negative correlation (−0.28) exists between rating and review length.

• Engagement metrics show moderate to strong inter-correlations (up to 0.65).

• Lower-rated reviews tend to receive slightly higher average useful votes.

• Average rating stabilizes over time between 3.7 and 4.1 stars.

•Text-based analysis revealed consistent beauty-service themes and positive sentiment expressions across reviews.

Challenges:

Extreme outliers in review length and useful votes required careful visualization and interpretation. Multiple plot types were used to avoid misleading conclusions and ensure accurate pattern identification.